

Vision Transformer (ViT) with combination of SROIE, CORD and Wild Receipt Datasets

Manraj Singh

The University of Texas at Dallas
mxs220007@utdallas.edu

Aditya Samir Desai

The University of Texas at Dallas
asd210004@utdallas.edu

Sahil Rajiv Sheth

The University of Texas at Dallas
srs210011@utdallas.edu

Abstract: This project is a Vision Transformer (ViT) encoder model to provide well-structured information from the ICDAR 2019 SROIE, CORD, and Wild Receipt datasets [1] [16] [17]. Our model tries to predict the *company information, transaction date, total price, and address* from each data image in a 224x224 image. We implemented two different ViT configurations that are measured against a *Tesseract OCR plus regex baseline*. After training the model, we see that the ViT more than doubled the baseline's overall match score - going from 0.2860 to 0.5965. The most significant gain was seen in the address field. We applied temperature scaling to each field which calibrated the model's confidence, where all the fields returned a value greater than 1.0 indicating slight overconfidence which was consistent to our suspicions due to limiting factors such as a small dataset that isn't as reliable for constructing a "state of the art" ViT. The calibrated confidence scores were used to implement a Human-in-the-Loop (HITL) system with a program interface designed by modern full stack technologies such as React and python's fastapi service. HITL system, having a review system with field thresholds tuned to 95% precision, auto accepting roughly 50% of the test receipts without human intervention. Throughout the development of this model, we understood that there was a limitation in the closed-vocabulary classification approach that was implemented.

Keywords: Human-in-the-loop, Vision Transformer, Character recognition, receipt scanning, SROIE, WildReceipt, Cord.

I. Introduction

The overall goal in our Receipt digitization program is to provide a well-structured output, such as vendor information, transaction date, total price, and address, from a given image of a printed retail receipt. Success in implementing an accurate model opens the door to an expansive use case, such as: personal finance, business accounting, and/or tax auditing. However, one thing to note is while working with a large dataset, we will encounter a myriad of problems such as: low print quality images, crumpled paper leading to text distortion, disruptions from text outside of the receipt space, and irregular layouts.

Selecting the datasets removed such impurities and eased the path in implementing a model. For instance, the SROIE dataset consists of a four-field extraction benchmark with over 900 annotation images of receipts in the English language - collected from various Malaysian retailers [1].

II. Preprocessing

The program pipeline goes as follows: load the dataset, partition the data, image processing, vocabulary construction, model development, training, logging, and recalibration. The dataset is fetched via

an API call to the Huggingface repository that hosts the dataset. This is followed by pairing each receipt image (in JPG format) to the matching JSON file. We then join the datasets together in a pandas data frame where each row displays the file name, pixel width/height, and its designated labels (vendor, date, total price, and address) - empty entities are defaulted to NULL. The train-test datasets come presplit, so we further split them for train/validation/test operations with a respective 80/10/10 ratio - for clarification the datasets are randomly rearranged.

The image preprocessing consists of converting each RGB image to a 3-channel grayscale, resizing to 224x224 pixels, transform to tensor, and normalizing the images with a mean set of 0.485, 0.456, and 0.406, and a standard deviation set of 0.229, 0.224, and 0.225 which derives from the ImageNet training set [2] which was used by the PyTorch torchvision library [3]. For training sets, we implemented a random rotation of 3 degrees and a color jitter with a brightness and contrast factor of 0.3 to mitigate overfitting and technically providing the model with a new dataset at each run interval.

Sample Training Receipts

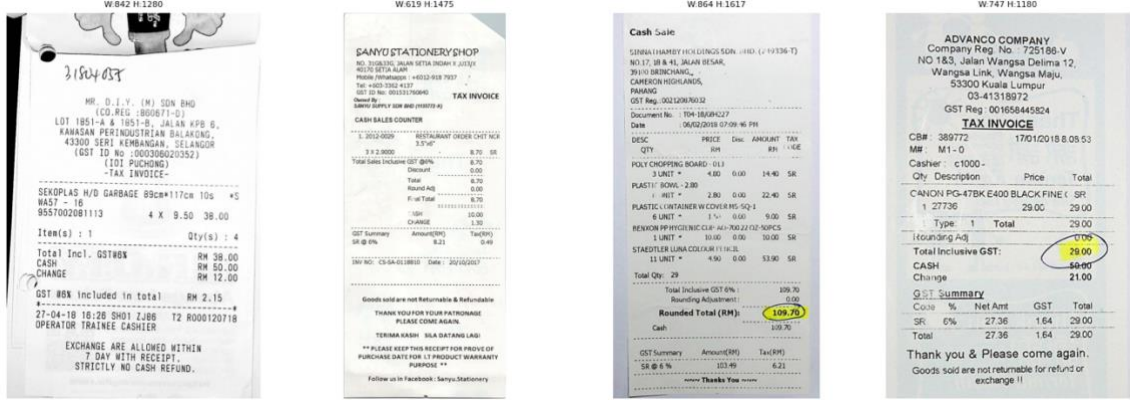


Fig. 2. EDA Sample of Four images in preprocessing

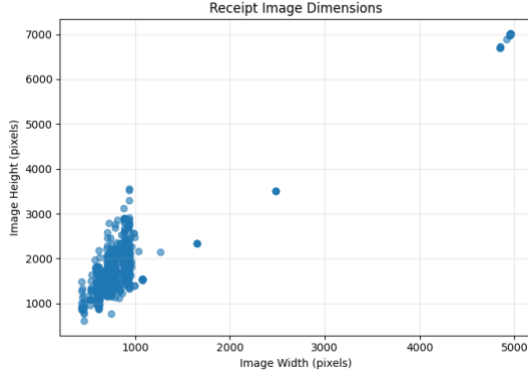


Fig. 2 Dimensions of Receipt before preprocessing

Furthermore, we utilize the dataset to produce a closed vocabulary where each label is normalized by a field type - total prices rounded to nearest hundredths place, dates are formatted to DD/MM/YYYY, and addresses have extra whitespaces removed. The vocab is saved in a .JSON file and becomes the reference sheet the model uses to match testing data with its corresponding string from the .JSON file.

III. Vision Transformer

A) Patch Embedding

The ViT encoder operates on a preprocessed receipt image $x \in \mathbb{R}^{C \times H \times W}$, where C is the number of channels and H and W represent the height and width respectfully. We chose a patch size of $P = 16$ (i.e. 16×16 square), then decomposed the image to $N = (224/16)^2 = 196$ non-overlapping $P \times P$ patches. Each patch x_i (where $i = 1, \dots, N$) is vectorized for the input layer which will be denoted as: $x_i \in \mathbb{R}^{3 \times 16 \times 16}$. In turn, each patches' vectorized input layer for the transformer at x_i equals 768 (i.e. the number of raw

pixel values in one patch when flattened to a single vector). [4]

Every vectorized patch is mapped to a D -dimensional embedding space, where the weights represent the linear transformation + the bias (representing the shift):

$$z_{0,i} = W_p x_i + b_p \quad (1)$$

Following patch embedding formula of Dosovitskiy et al. [4].

Where $z_{0,i}$ represents the embedded representation of each patch from $0, \dots, i$, W_p is the weight matrix with respect to each patches' vectorized input layer, x_i being the vectorized input layer as stated before, and b_p being the bias vector with respect to the dimension.

This, in turn, projects each patch from 768 to the model's embedding dimension of $D = 256$ [4] [5]. The weight matrix is initialized via a Kaiming Uniform Distribution which submitted the best results in stabilizing gradient flow for the first training step. [6] [7]

B) Learnable Positional Embedding

Since the transformer input takes in all 196 patch tokens all at once, it will not know the position of the patch from the deconstructed receipt image. The positional embedding tensor:

$$E_{pos} \in \mathbb{R}^{1 \times N \times D} \quad (2)$$

Equation adapted from Dosovitskiy et al. [4].

Where the leading dimension 1 is for the positional information consists of a single set with shape $(1, 196, 256)$. Information on the position goes

into z'_0 in turn updating the embedded representation of each patch with its position:

$$z'_0 = z_0 + E_{pos} \quad (3)$$

Equation for updating patch embeddings with position, adapted from Dosovitskiy et al. [4].

All signatures will be initialized randomly with a normal distribution. [7]

C) Multi-Headed Self Attention

This is where each patch can essentially communicate with other patches to work together to make connections inside the dataset. An input sequence $z \in \mathbb{R}^{B \times N \times D}$, and a single fused linear projection $W_{QKV} \in \mathbb{R}^{(D \times 3D)}$ both produce queries Q, keys K, and values V. These are reconfigured to H independent heads of dimension $d = D/H$ over which scaled dot-product attention (A) is computed in parallel: [7]

$$Attention(QKV) = softmax(\frac{QK^T}{\sqrt{D_h}})V \quad (4)$$

This equation, following formula of Dosovitskiy et al. [4], pushes the SoftMax method while keeping values in a stable range.

The outputs are passed through an output projection – restoring the embedding dimension $D = 256$. Dropout is applied to both the attention weights and the projected output which helps reduce overfitting. We ensure that the dimension D is divisible by the number of heads, ensuring that there is an even distribution.

D) Feed-Forward Block & Pre-Norm Encoder

Every encoding layer works by performing self-attention (patch getting info from other patches) and feed-forward. Feedforwarding is essentially implementing a small neural network for each patch to allow it to learn what it got from self-attention. The Feed-forward block is a two-layer MLP (multi-layer perceptron)[8] consisting of $4 \times 256 = 1024$ hidden nodes (4 being a chosen constant for expansion, 256 being the token size) [8], a GELU non-linearity[9], and two dropout methods [10] - one after activation and one after compressing back to 256.

E) Encoder Stack and Field Classification

Once the FeedForward and Pre-Normalization Encoder layer is complete, it passes through the encoder stack where each pass gives the patches another chance to share and refine information with the other patches. When the tokens exit the last layer, each of the 196 patches don't represent its own 16×16

region of the receipt but the context for understanding the whole receipt [4]. There are four field classification heads to determine the prediction categories (vendor, date, total price and address). Each head averages out the 196 tokens into a 256-dimension vector that provides a summary of the receipt. The summary is passed to a two-layer neural network which compresses the 256 dimension vector to 128 with a GELU activation, its then expanded to a set of output nodes which consists of the number of labels in the field's closed vocabulary (for instance 222 for vendor, 322 for date, 415 for total price and 267 for address) [9]. Each output node produces a score showing how confident the model believes the receipt matches with the label. The final prediction for each field is the label with the highest confidence score.

IV. Model Implementation & Confidence Scoring

A) Model Implementation

The model was trained over 80 epochs with a batch size of 64 and a learning rate of 1×10^{-3} using the Pytorch AdamW optimizer [11] which performed well for our small dataset but was still computationally expensive. In order to prevent a spike in gradient updates, we implemented a gradient normalization and clipping across all parameters with a maximum normalization value of 1.0.

We first run a warm-up over the first 10 epochs which consistently increases, we then transition to cosine decay for the last 35 epochs which prevents the model from making asymptotically large updates in the initial parts of training. The training loss is computed by the average cross-entropy across the four prediction categories: [13]

$$loss = \frac{1}{Fields(F)} \sum_{f=1}^F CrossEntropy(\hat{y}_f, y_f) \quad (5)$$

Where $Fields(F) = 4$ are averaged, maintaining a stable gradient magnitude during training, influenced by [11]

By averaging, we mitigated the value of the gradient which caused training accuracy to drop below 10%. Upon updating, we saw a smaller drop in loss in the first epoch - in turn increasing accuracy to a favorable amount.

B) Confidence Scoring

Since model output scores will not fully reflect on how accurate/confident it was on predicting the four categories, we applied temperature scaling across the four categories. This is performed by initializing the temperature parameter (T) which is a value greater than zero across the four categories. Then, we put the

raw model scores over T and implement SoftMax on that value. [14]

$$Prob_f = softmax\left(\frac{y_f}{T_f}\right) \quad (6)$$

Where y_f are the logits at the end of the model over its temperature scalar T_f at the end of the model – adapted for code implementation from Guo et al. [14]

With this, a temperature value above 1.0 flattens (i.e. overconfident), while below 1.0 increases the confidence value. In the instances where the model is over-confident, we calibrate it using the Expected Calibration Error (ECE) which measures the weighted average gap between the output confidence vs the actual accuracy across all categories [14]:

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |acc(B_m) - conf(B_m)| \quad (7)$$

ECE formula calculates the summation over the number of bins M of the absolute difference between the legitimate accuracy minus confidence prediction of each prediction that is in a certain confidence level range (bin B) multiplied by the weight of the bin - from Guo et al. [14]

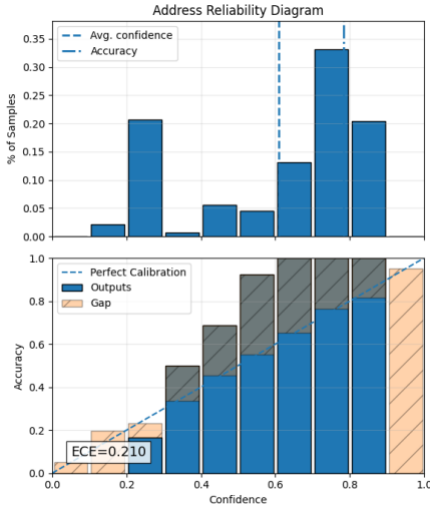


Fig. 3. Confidence Scoring across Address

V. Human in the Loop Implementation

Even with calibration along every step of the way, we will not receive 100% accurate predictions on every single field. Instead of accepting this fact, we allow the user to select the set of unconfident predictions for verification. All of the corrections made by the user will get updated into the log.

The way we do that is after temperature scaling creates its probability values for each field, we iterate over each confidence score in the validation set and check for one that approaches 95% precision on each field [15].

In addition, a single passing threshold is found in each field, the receipt is going to be auto-accepted whenever all of its four fields meet the threshold – therefore, if one field fails to meet the threshold, it goes in for review.

Fig. 4. User interface for verifying features

VI. Results

Field	Baseline Exact Match	Baseline F1	Vit Exact Match	ViT F1
Company	0.1297	0.3319	0.6224	0.6224
Date	0.7061	0.7080	0.6224	0.6224
Address	0.0115	0.5350	0.6224	0.6224
Total Price	0.2968	0.3007	0.5190	0.5190
Overall	0.2860	0.4689	0.5965	0.5965

Fig. 5. Results Captured in experiment1_results.json (Baseline) against the ViT results in Experiment 3

The ViT model performs better than the baseline in every single field except date possibly due to the regular expression matching being very consistent. Where we saw the most improvement was in address going from a near-zero base line to 0.6224. This shows the ViT can captures better relationships between patches within the receipt image, compared to the baseline regular expression which is seen to poorly reconstruct them. In turn, we can conclude that our positional embedding process has performed better than the baseline regular expression process specifically when it comes placing the tokens in the correct order.

One important thing to look at is the similar values in ViT Exact Match and ViT F1. Those can be interpreted as the ViT model hitting a limitation due to the closed vocabulary in place that references the training dataset. If more training data was added, there will most certainly be a higher threshold met for these values – i.e. more values in the close vocabulary.

Field	Experiment
Company	0.7828
Date	0.7828
Address	0.7828
Total Price	0.4448
Overall	0.6983

Fig. 6. ViT Validation Accuracy from the best performing epoch

The ViT model had an overall best accuracy of 0.6983, noting that company, date and address had the same accuracy measurements. This can be due to the positional embedding weighing heavy on the first three fields and accurately matching them to the vocabulary. This can be explained as the vocabulary having a less limited set of combinations as compared to a field like total price which is free to have different combinations.

Field	Threshold
Company	0.55
Date	0.55
Address	0.35
Total Price	0.45

Fig. 7. Thresholds for human-in-loop

The following thresholds listed above come from the implementation of equation 6 in our confidence scoring section. We see that fields company and date have the highest threshold value of 55%, this is reasonable since these fields have performed well across the other tables.

However, unusually, address carries the lowest threshold at 35% only if you refer to the ViT’s validation accuracy table. If we take a more holistic approach, we see that this makes sense due to the drastic increase from baseline. In turn, we are expected to see a lot of receipts unreasonably delivered to the user for verification. Total price is at around 45% which reflects on the lower confidence seen for this field – this field is consistent with its weak performance in both validation and test evaluations.

Field	Temperature
Company	1.2662
Date	1.0322
Address	1.1375
Total Price	1.2080

Fig. 8. Calibration Temperatures from model, derived from Section IV-B

One of the metrics that didn’t wasn’t a surprise, all of our fields had a temperature metric that was above 1, meaning our model prediction head is overconfident. However, the SoftMax method implemented in equation 6 theoretically flattens the distribution and produces the calibrated confidence scores. In turn, our temperature scores calibrate our confidence scores, which compare against our threshold values, which directly affects the frequency of human verification.

Overall, the results show that the ViT model is a better approach compared to our baseline which uses regular expressions – you can simply say it is more than twice as accurate. However, limitations in the closed vocabulary and small dataset size are more pronounced at the end of the model’s pipeline – with this being a computationally expensive model, we are limited by the amount of times we could test it with respect to the deadline.

VII. Conclusion

This project demonstrates that Vision Transformers can be created from scratch and use receipt image data to create structured information. The Vision Transformer was shown to perform at least as well or better than a traditional optical character recognition-based system on unformatted, variable-length fields such as Vendor Name and Address. However, when rules are strictly followed, and variability is limited (as with a date) the Tesseract System will continue to perform better. In addition, while a Rule-Based System is better suited for documents with a standard layout, the Vision Transformer uses global spatial knowledge learned during training to interpret documents that do not follow a standard layout without requiring custom regular expression code.

The successful application of per-field temperature scaling to calibrate the model’s confidence allowed it to accurately produce a reliable estimate of probability for each prediction. Calibration

in this fashion enabled the selective-prediction system to gate low-confidence predictions into a Human-in-the-Loop (HITL) review queue. Over 51% of high-confidence receipts were automatically accepted by the HITL queue during testing. The results clearly showed that an operational path exists to automate full receipt scanning while maintaining quality control.

The current closed-vocabulary approach to classification creates a performance bottleneck. Since the model can only make predictions about labels that have been presented during training, the test set's F1 score cannot exceed a certain threshold and can be significantly affected by new (unknown) merchants and addresses; therefore, future research will investigate using a hybrid character-based or LayoutLM-type architecture to predict against previously unseen text but still maintain the spatial reasoning capabilities of the transformer's encoder; also, by using pre-trained weights from much larger document data sets to initialize the encoder would help improve the robustness of the model in very noisy environments that are typical for photographing receipts.

VIII. References

- [1] Z. Huang et al., "ICDAR2019 Competition on Scanned Receipt OCR and Information Extraction," in Proc. 15th Int. Conf. Document Anal. Recognit. (ICDAR), Sydney, Australia, Sep. 2019, pp. 1516–1520. Accessed: May 1, 2026.
- [2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in 2009 IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Miami, FL, USA, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848. Accessed: May 1, 2026.
- [3] "How to use CutMix and MixUp," Torchvision Documentation. [Online]. Available: https://docs.pytorch.org/vision/main/auto_examples/transforms/plot_cutmix_mixup.html Accessed: May 1, 2026.
- [4] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," arXiv preprint arXiv:2010.11929, Oct. 2020. [Online]. Available: <https://arxiv.org/pdf/2010.11929> Accessed: May 1, 2026.
- [5] ExplainingAI, "PATCH EMBEDDING | Vision Transformers explained," YouTube, Oct. 20, 2023. [Online]. Available: <https://www.youtube.com/watch?v=IBicvB4iyYU> Accessed: May 1, 2026.
- [6] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, "11.8. Transformers for Vision," in Dive into Deep Learning. [Online]. Available: https://d2l.ai/chapter_attention-mechanisms-and-transformers/vision-transformer.html Accessed: May 1, 2026.
- [7] "nn.init," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/2.11/nn.init.html> Accessed: May 1, 2026.
- [8] "Sequential," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/2.11/generated/torch.nn.Sequential.html> Accessed: May 1, 2026.
- [9] "GELU," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/2.11/generated/torch.nn.GELU.html> Accessed: May 2, 2026.
- [10] "Dropout," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/2.11/generated/torch.nn.Dropout.html#torch.nn.Dropout> Accessed: May 2, 2026.
- [11] "AdamW," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/main/generated/torch.optim.AdamW.html> Accessed: May 2, 2026.
- [12] "clip_grad_norm_," PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/docs/2.11/generated/torch.nn.utils.clip_grad_norm_.html Accessed: May 2, 2026.
- [13] "CrossEntropyLoss," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/main/generated/torch.nn.CrossEntropyLoss.html> Accessed: May 3, 2026.

- [14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70, Sydney, Australia: PMLR, Aug. 2017, pp. 1321-1330. [Online]. Available: <https://proceedings.mlr.press/v70/guo17a/guo17a.pdf> Accessed: May 3, 2026.
- [15] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, I. Guyon et al., Eds. Curran Associates, Inc., 2017, pp. 5998-6008. Accessed: May 3, 2026.
- [16] NAVER CLOVA, "cord-v2," Hugging Face Datasets. [Online]. Available: <https://huggingface.co/datasets/naver-clova-ix/cord-v2>. Accessed: May 3, 2026.
- [17] Alibaba Research, "AdvancedLiterateMachinery - OCR," GitHub. [Online]. Available: <https://github.com/AlibabaResearch/AdvancedLiterateMachinery/tree/main/OCR>. Accessed: May 3, 2026.